

that the bug was a `NullPointerException` in the `org.eclipse.jface.dialogs.DialogSettings.load` method in `DialogSettings.java` in line 278. The stack trace had 56 method calls in it, starting from the `org.eclipse.equinox.launcher.Main.run` method in `Main.java` line 1236. If I were to fix this bug as a developer, it would be so difficult to just reproduce this problem by creating a test case on my own. Further, it would be close to impossible to ensure that whatever fix I made was correct... and that I had not broken any existing functionality. Obviously, Eclipse is a huge code-base, and there is a large testing infrastructure for Eclipse. Still, any developer who has worked on such large code-bases will understand what I said—the complexity of testing in such code-bases is overwhelming. In other words, the enormous complexity of the software renders testing ineffective, no matter what testing technique, method or process you use.

From another perspective, the bugs found during testing are only the tip of the iceberg: For every bug found in testing, there are 10 or more bugs lurking beneath yet to be uncovered! In this way, software bugs are akin to icebergs—what we see is the tip of the iceberg, while 1/8th to 1/10th of the iceberg is submerged beneath the water. In other words, these latent bugs lurk in the darkness, only to be exposed later when changes are made to the software, or when a user attempts to use some unusual functionality. For certain kinds of bugs such as those that are data-flow-based or functionality-based, testing is effective. For other kinds such as design bugs, testing is ineffective.

To summarise, focusing extensively on testing is not the right strategy to ensure the high quality of the software. The alternative has been well-known for many decades. But what is it?

There is no silver bullet for creating high-quality software, but there is one solution that comes close. It is the lowly and humble manual review! For a long time, it has been well-acknowledged that manual review techniques such as peer-reviews, inspections, code and design walkthroughs are effective in creating high-quality software. There are many reasons why manual reviews are more effective than testing. First, manual reviews not only find actual bugs, they also find potential and latent bugs. They can also unearth hard-to-find bugs early in the software development life-cycle, such as design bugs (this fact is important because luminaries such as Capers Jones found that 25 to 64 per cent of bugs in software are design bugs). Further, reviews don't just find bugs, they also identify weak spots in software where improvements are required, and hence help improve software quality.

Though manual reviews are used in the software industry, they haven't received the focus and attention they deserve. If manual reviews are so useful, why aren't they used extensively? The main reason is that they are effort-intensive.

And because of this, most development projects, hurrying to meet deadlines, skip manual reviews and depend on testing to catch the bugs. So what can we do about this? Fortunately, static analysers can address this limitation of manual reviews.

Starting from the days of simple 'lint'-like pattern matchers, static analysers have come a long way. Today, there are sophisticated static analysers that use modern and advanced techniques such as model checking, abstract interpretation and program querying. These tools are able to find bugs that are hard to detect with testing, or even manual reviews. These tools also scale well to work on large software implementations, and find bugs in millions of lines of code with ease. Though many of these tools are costly, since they spot the hard-to-find bugs, the ROI (return-on-investment) is very good. Research shows that a considerable percentage of bugs that are detectable by manual reviews can be automatically detected using static analysers.

By integrating static analysers as part of the software development life-cycle, bugs can be found earlier. After that, if extensive manual review is performed on the code, reviewers can focus on harder-to-find bugs. With extensive static analysis and manual reviews, testing will find only a few bugs (mostly those that relate to interface and context-related problems, since they are not well addressed by manual reviews and static analysers).

Consider the two examples I mentioned in the beginning of this article. The call to a pure virtual function inside a constructor is a bug that can be easily caught using both static analysis tools as well as manual reviews. However, only if we create the right test-cases, and if the bug manifests in the given context, can testing find this bug; otherwise, it wouldn't. For the problem in the event-handling code, manual reviews can help find it, since it is likely to be a design problem. Static analysers are not (yet) effective in finding such problems. Advanced testing techniques, such as model-based testing (MBT), can help find such bugs, but MBT is not widely used in practice.

When manual reviews are combined with static analysis tools, and are made an integral part of the development life-cycle, they can help create high-quality software. Of course, testing has an important role to play, for example, in acceptance testing. However, the right strategy for creating high-quality software is to focus extensively on manual reviews and static analysis, in addition to testing the software. 

By: S G Ganesh

The author works for Siemens (Corporate Research & Technologies), Bangalore. You can reach him at sgganesh@gmail.com.